



Clase 44 - Docker: comandos principales

1. Contexto de trabajo

Esta primera clase parte del entorno preparado previamente:

- **Windows** como equipo del estudiante.
- **Visual Studio Code** instalado en Windows.
- Extensión **Remote - SSH** de VS Code.
- **Ubuntu Server 24.04** ejecutándose en una máquina virtual.
- Conexión desde VS Code a Ubuntu mediante **SSH**.
- **Docker Engine** instalado en Ubuntu Server.
- El usuario de Ubuntu pertenece al grupo `docker` , por lo que puede ejecutar Docker sin `sudo` .
- Acceso a Internet desde Ubuntu para descargar imágenes desde Docker Hub.

Importante: todos los comandos de Docker de esta clase se ejecutan en la **terminal remota de Ubuntu Server dentro de VS Code**, no en PowerShell ni en CMD de Windows.

2. Antes de comenzar

2.1 Conectarse a Ubuntu Server desde VS Code

En VS Code:

1. Pulsa:

```
Ctrl + Shift + P
```

1. Busca:

```
Remote-SSH: Connect Current Window to Host...
```

1. Selecciona el servidor Ubuntu configurado anteriormente.
2. Introduce la contraseña si se solicita.
3. Comprueba que en la esquina inferior izquierda aparece algo similar a:

```
SSH: 192.168.x.x
```

La dirección IP exacta puede ser diferente en cada equipo.

2.2 Abrir la terminal de Ubuntu

En VS Code:

```
Terminal > New Terminal
```

Comprueba primero dónde estás trabajando:

```
whoami
```

```
hostname
```

```
uname -a
```

```
pwd
```

Estos comandos no son de Docker. Nos permiten verificar que estamos trabajando realmente en el servidor Ubuntu remoto.

3. Comprobar Docker

3.1 Ver la versión instalada

```
docker --version
```

Ejemplo de salida:

```
Docker version 29.x.x, build ...
```

¿Qué comprueba este comando?

Comprueba que el cliente de Docker está instalado y disponible en el sistema.

3.2 Ver información general de Docker

```
docker info
```

Este comando muestra información como:

- versión del servidor Docker;
- número de contenedores;
- número de imágenes;
- sistema de almacenamiento;
- sistema operativo;
- arquitectura;
- número de CPU;
- memoria disponible.

No es necesario comprender todavía todos los campos.

3.3 Comprobar el servicio Docker

```
systemctl status docker
```

Debería aparecer:

```
Active: active (running)
```

Para salir de la pantalla de `systemctl` :

```
q
```

4. Dos conceptos fundamentales: imagen y contenedor

Antes de practicar los comandos es necesario distinguir estos dos conceptos.

4.1 Imagen Docker

Una **imagen** es una plantilla preparada para crear contenedores.

Puede contener:

- un sistema base;
- una aplicación;
- librerías;
- dependencias;
- configuraciones.

Ejemplos de imágenes:

```
nginx
ubuntu
alpine
postgres
mysql
python
redis
```

Podemos imaginar una imagen como un **molde**.

4.2 Contenedor Docker

Un **contenedor** es una instancia ejecutándose a partir de una imagen.

Si la imagen es el molde, el contenedor es el objeto creado utilizando ese molde.

Una misma imagen puede utilizarse para crear muchos contenedores.

Ejemplo:

```
Imagen nginx
|
+-- contenedor web1
+-- contenedor web2
+-- contenedor web3
```

5. Sintaxis básica de Docker

La estructura general de muchos comandos es:

```
docker <comando> <opciones> <objeto>
```

Ejemplos:

```
docker ps
```

```
docker pull nginx
```

```
docker stop web1
```

```
docker rm web1
```

7. **docker pull** — descargar una imagen

7.1 Descargar Nginx

Ejecuta:

```
docker pull nginx
```

Docker descargará la imagen desde Docker Hub.

7.2 ¿Qué acaba de ocurrir?

Docker ha realizado, de forma simplificada, lo siguiente:

```
Ubuntu Server
  |
  | docker pull nginx
  v
Docker Hub
  |
  | descarga
  v
Imagen nginx almacenada localmente
```

7.3 Descargar una versión concreta

También podemos indicar una etiqueta o **tag**:

```
docker pull nginx:alpine
```

Aquí:

- **nginx** es el nombre de la imagen;
- **alpine** es el tag.

Cuando no especificamos un tag:

```
docker pull nginx
```

Docker utiliza normalmente:

```
latest
```

Es equivalente a:

```
docker pull nginx:latest
```

8. docker images — ver las imágenes locales

Ejecuta:

```
docker images
```

También puede usarse:

```
docker image ls
```

La salida contiene columnas parecidas a:

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
nginx	latest	...	...	...
nginx	alpine	...	...	...

Campos importantes

- **REPOSITORY**: nombre de la imagen.
- **TAG**: versión o variante.
- **IMAGE ID**: identificador de la imagen.
- **SIZE**: espacio ocupado.

9. docker run — crear y ejecutar un contenedor

Este es uno de los comandos más importantes de Docker.

La forma general es:

```
docker run [opciones] imagen
```

9.1 Primer ejemplo: hello-world

Ejecuta:

```
docker run hello-world
```

Si todo funciona correctamente aparecerá:

```
Hello from Docker!
```

¿Qué hace Docker?

Cuando ejecutamos:

```
docker run hello-world
```

Docker:

1. busca la imagen `hello-world` en el equipo;
2. si no existe, la descarga;
3. crea un contenedor;
4. ejecuta el programa del contenedor;

- 5. el programa muestra el mensaje;
- 6. el contenedor finaliza.

10. `docker ps` — ver contenedores

10.1 Ver contenedores en ejecución

```
docker ps
```

Solo muestra los contenedores actualmente activos.

10.2 Ver todos los contenedores

```
docker ps -a
```

La opción:

```
-a
```

significa:

```
all
```

y muestra tanto contenedores en ejecución como detenidos.

10.3 Campos importantes

Una salida típica contiene:

```
CONTAINER ID
IMAGE
COMMAND
CREATED
STATUS
PORTS
NAMES
```

Los más importantes al principio son:

- **CONTAINER ID:** identificador del contenedor.
- **IMAGE:** imagen utilizada.
- **STATUS:** estado.
- **PORTS:** puertos publicados.
- **NAMES:** nombre del contenedor.

11. Ejecutar un servidor web Nginx

Ahora crearemos un contenedor que permanecerá funcionando.

Ejecuta:

```
docker run -d --name nginx-prueba -p 80:80 nginx
```

Vamos a estudiar el comando por partes.

11.1 `docker run`

```
docker run
```

Crea un nuevo contenedor y lo ejecuta.

11.2 `d`

```
-d
```

Significa **detached mode**.

El contenedor se ejecuta en segundo plano y la terminal queda libre.

Sin `-d`, algunos programas se quedan ocupando la terminal.

11.3 `-name nginx-prueba`

```
--name nginx-prueba
```

Asigna un nombre al contenedor.

En vez de trabajar con un identificador como:

```
7f46cbe2a41d
```

podemos utilizar:

```
nginx-prueba
```

11.4 `p 80:80`

```
-p 80:80
```

Publica un puerto.

La estructura es:

```
-p PUERTO_HOST:PUERTO_CONTENEDOR
```

Por tanto:

```
-p 80:80
```

significa:

```
Puerto 80 de Ubuntu Server
  |
  v
Puerto 80 del contenedor
```

Nginx escucha dentro del contenedor en el puerto `80`.

11.5 `nginx`

La última parte:

```
nginx
```

indica la imagen que Docker utilizará para crear el contenedor.

12. Comprobar que Nginx está funcionando

Ejecuta:

```
docker ps
```

Deberías ver:

```
nginx-prueba
```

con un estado parecido a:

```
Up ...
```

12.1 Obtener la IP del servidor

```
hostname -I
```

Ejemplo:

```
192.168.86.130
```

12.2 Probar desde Windows

En el navegador de Windows abre:

```
http://IP_DE_UBUNTU
```

Ejemplo:

```
http://192.168.86.130
```

Si aparece la página:

```
Welcome to nginx!
```

significa que:

- Docker funciona;
- el contenedor está ejecutándose;
- Nginx funciona;
- la publicación de puertos funciona;
- Windows puede comunicarse con Ubuntu Server.

13. `docker stop` — detener un contenedor

Ejecuta:

```
docker stop nginx-prueba
```

Comprueba:

```
docker ps
```

El contenedor ya no aparecerá.

Ahora ejecuta:

```
docker ps -a
```

El contenedor seguirá existiendo, pero estará detenido.

14. **docker start** — iniciar un contenedor detenido

Ejecuta:

```
docker start nginx-prueba
```

Comprueba:

```
docker ps
```

El contenedor vuelve a estar en ejecución.

15. **docker restart** — reiniciar un contenedor

Ejecuta:

```
docker restart nginx-prueba
```

Este comando equivale conceptualmente a:

```
detener + volver a iniciar
```

Es útil cuando queremos reiniciar rápidamente un servicio.

16. **docker logs** — consultar los logs

Ejecuta:

```
docker logs nginx-prueba
```

Nginx mostrará las peticiones recibidas.

Si has abierto la página desde el navegador, probablemente aparezcan líneas similares a:

```
GET / HTTP/1.1
```

16.1 Seguir los logs en tiempo real

```
docker logs -f nginx-prueba
```

La opción:

```
-f
```

significa **follow**.

Ahora vuelve a abrir o refrescar la página Nginx en el navegador.

Verás nuevas entradas en la terminal.

Para dejar de seguir los logs:

```
Ctrl + C
```

Esto no detiene el contenedor.

17. **docker exec** — ejecutar comandos dentro de un contenedor

docker exec permite ejecutar un comando dentro de un contenedor que ya está funcionando.

Ejemplo:

```
docker exec nginx-prueba hostname
```

El resultado mostrará el nombre de host interno del contenedor.

17.1 Abrir una shell dentro del contenedor

Ejecuta:

```
docker exec -it nginx-prueba /bin/bash
```

Aquí:

```
-i
```

mantiene abierta la entrada estándar.

```
-t
```

crea una terminal interactiva.

En conjunto suele escribirse:

```
-it
```

Ahora estamos dentro del contenedor.

Comprueba:

```
hostname
```

```
pwd
```

```
ls
```

Para salir:

```
exit
```

El contenedor seguirá funcionando.

18. `docker inspect` — ver información detallada

Ejecuta:

```
docker inspect nginx-prueba
```

La salida está en formato JSON y contiene mucha información.

Por ahora interesa saber que permite consultar:

- configuración;
- red;
- IP interna;
- puertos;
- volúmenes;
- estado;
- variables;
- imagen utilizada.

19. `docker stats` — ver consumo de recursos

Ejecuta:

```
docker stats
```

Muestra en tiempo real información como:

- CPU;
- memoria;
- tráfico de red;
- operaciones de entrada/salida.

Para salir:

```
Ctrl + C
```

También podemos consultar un solo contenedor:

```
docker stats nginx-prueba
```

20. `docker port` — consultar puertos publicados

Ejecuta:

```
docker port nginx-prueba
```

Podría mostrar algo parecido a:

```
80/tcp -> 0.0.0.0:80
```

Esto indica que el puerto `80` del contenedor está publicado en el servidor.

21. Crear varios contenedores de la misma imagen

Una imagen puede crear múltiples contenedores.

Primero mantendremos:

```
nginx-prueba
```

en el puerto `80`.

Ahora crea otro:

```
docker run -d --name nginx-segundo -p 8080:80 nginx
```

Comprueba:

```
docker ps
```

Ahora deberían existir dos contenedores Nginx.

Desde Windows:

```
http://IP_DE_UBUNTU
```

y:

```
http://IP_DE_UBUNTU:8080
```

Los dos utilizan la misma imagen, pero son contenedores diferentes.

22. ¿Por qué no podemos usar el mismo puerto dos veces?

Prueba:

```
docker run -d --name nginx-tercero -p 80:80 nginx
```

Debería producirse un error porque el puerto `80` de Ubuntu ya está ocupado por `nginx-prueba`.

La solución es utilizar otro puerto del host:

```
docker run -d --name nginx-tercero -p 8081:80 nginx
```

Ahora podremos acceder mediante:

```
http://IP_DE_UBUNTU:8081
```

23. `docker rm` — eliminar contenedores

Un contenedor debe estar detenido antes de eliminarlo normalmente.

Detén:

```
docker stop nginx-segundo
```

Elimina:

```
docker rm nginx-segundo
```

Comprueba:

```
docker ps -a
```

23.1 Forzar la eliminación

También existe:

```
docker rm -f nginx-segundo
```

La opción `-f` fuerza la eliminación incluso si está en ejecución.

En esta primera clase es preferible aprender primero a detener el contenedor explícitamente con `docker stop`.

24. `docker rmi` — eliminar imágenes

Para eliminar una imagen:

```
docker rmi nginx:alpine
```

También puede utilizarse:

```
docker image rm nginx:alpine
```

Comprueba después:

```
docker images
```

Docker no permitirá eliminar normalmente una imagen si todavía existen contenedores que dependen de ella.

25. `-rm` — eliminar automáticamente un contenedor al terminar

Ejecuta:

```
docker run --rm hello-world
```

Cuando el programa termine, Docker eliminará automáticamente el contenedor.

Es útil para contenedores temporales.

26. `docker run -it` — contenedores interactivos

Descarga y ejecuta Ubuntu:

```
docker run -it --name ubuntu-prueba ubuntu bash
```

Ahora estarás dentro de un contenedor Ubuntu.

Comprueba:

```
cat /etc/os-release
```

```
hostname
```

```
pwd
```

```
ls
```

Sal:

```
exit
```

Después comprueba:

```
docker ps -a
```

El contenedor aparecerá detenido.

27. Diferencia entre `run`, `start` y `exec`

Esta diferencia es fundamental.

`docker run`

```
docker run ...
```

Crea **un contenedor nuevo** y lo inicia.

Ejemplo:

```
docker run -d --name web1 nginx
```

`docker start`

```
docker start ...
```

Inicia **un contenedor que ya existe**.

Ejemplo:

```
docker start web1
```

`docker exec`

```
docker exec ...
```

Ejecuta **un comando dentro de un contenedor que ya está funcionando**.

Ejemplo:

```
docker exec web1 hostname
```

Resumen:

Comando	¿Crea contenedor?	¿Necesita que exista?	Uso principal
<code>docker run</code>	Sí	No	Crear y arrancar
<code>docker start</code>	No	Sí	Arrancar uno detenido
<code>docker exec</code>	No	Sí	Ejecutar algo dentro de uno activo

28. `docker cp` — copiar archivos

Podemos copiar archivos entre Ubuntu Server y un contenedor.

Crea un archivo en Ubuntu:

```
echo "Hola desde Ubuntu Server" > saludo.txt
```

Cópialo al contenedor:

```
docker cp saludo.txt nginx-prueba:/tmp/saludo.txt
```

Comprueba el contenido dentro del contenedor:

```
docker exec nginx-prueba cat /tmp/saludo.txt
```

También puede copiarse en sentido contrario.

Ejemplo:

```
docker cp nginx-prueba:/tmp/saludo.txt saludo-copia.txt
```

29. Comandos de ayuda

Docker incluye ayuda integrada.

Ejecuta:

```
docker --help
```

Ayuda para un comando concreto:

```
docker run --help
```

```
docker ps --help
```

```
docker logs --help
```

Aprender a consultar `--help` es más importante que memorizar todas las opciones.

30. Resumen de comandos principales

Comando	Función
<code>docker --version</code>	Ver versión
<code>docker info</code>	Información del sistema Docker
<code>docker pull IMAGEN</code>	Descargar una imagen
<code>docker images</code>	Mostrar imágenes locales
<code>docker run IMAGEN</code>	Crear y ejecutar un contenedor
<code>docker run -d ...</code>	Ejecutar en segundo plano
<code>docker run --name ...</code>	Asignar nombre
<code>docker run -p HOST:CONTENEDOR ...</code>	Publicar un puerto
<code>docker ps</code>	Ver contenedores activos
<code>docker ps -a</code>	Ver todos los contenedores
<code>docker stop CONTENEDOR</code>	Detener
<code>docker start CONTENEDOR</code>	Iniciar uno detenido
<code>docker restart CONTENEDOR</code>	Reiniciar
<code>docker logs CONTENEDOR</code>	Consultar logs
<code>docker logs -f CONTENEDOR</code>	Seguir logs
<code>docker exec CONTENEDOR COMANDO</code>	Ejecutar un comando dentro

Comando	Función
<code>docker exec -it ... /bin/bash</code>	Abrir terminal dentro
<code>docker inspect CONTENEDOR</code>	Ver configuración detallada
<code>docker stats</code>	Ver consumo de recursos
<code>docker port CONTENEDOR</code>	Ver puertos publicados
<code>docker cp</code>	Copiar archivos
<code>docker rm CONTENEDOR</code>	Eliminar un contenedor
<code>docker rmi IMAGEN</code>	Eliminar una imagen
<code>docker --help</code>	Ayuda general

31. Esquema mental de trabajo con Docker

Un flujo típico es:



32. Ejercicios prácticos

A partir de este punto comienza la parte práctica.

La recomendación es realizar los ejercicios **sin copiar directamente la solución** y utilizar:

```
docker --help
```

o:

```
docker COMANDO --help
```

cuando sea necesario.

Ejercicio 1 — Verificar el entorno

Objetivo

Comprobar que el entorno está preparado.

Tareas

Ejecuta:

```
whoami
```

```
hostname
```

```
docker --version
```

```
docker info
```

```
docker ps
```

Preguntas

1. ¿Qué usuario estás utilizando?
2. ¿Cuál es el nombre del servidor?
3. ¿Qué versión de Docker tienes?
4. ¿Cuántos contenedores hay en ejecución?

Ejercicio 2 — Descargar imágenes

Objetivo

Practicar `docker pull` y `docker images`.

Tareas

Descarga:

```
hello-world
nginx
alpine
```

Después lista las imágenes disponibles.

Comandos que deberías necesitar

```
docker pull
```

```
docker images
```

Preguntas

1. ¿Qué imagen ocupa menos espacio?
2. ¿Qué tag aparece en cada imagen?
3. ¿Qué diferencia hay entre `REPOSITORY` y `IMAGE ID` ?

Ejercicio 3 — Primer contenedor

Objetivo

Crear un contenedor sencillo.

Ejecuta:

```
docker run hello-world
```

Después:

```
docker ps
```

y:

```
docker ps -a
```

Preguntas

1. ¿Aparece en `docker ps` ?
 2. ¿Aparece en `docker ps -a` ?
 3. ¿Por qué el contenedor no permanece ejecutándose?
-

Ejercicio 4 — Contenedor interactivo

Objetivo

Entrar en un contenedor.

Crea:

```
docker run -it --name alpine-lab alpine sh
```

Dentro del contenedor ejecuta:

```
hostname
```

```
pwd
```

```
ls
```

```
cat /etc/os-release
```

Después:

```
exit
```

Desde Ubuntu Server ejecuta:

```
docker ps -a
```

Preguntas

1. ¿Qué estado tiene `alpine-lab` ?
 2. ¿Qué ocurrió al ejecutar `exit` ?
 3. ¿Se eliminó el contenedor?
-

Ejercicio 5 — Volver a utilizar un contenedor existente

Objetivo

Diferenciar `run` y `start`.

Inicia:

```
docker start alpine-lab
```

Comprueba:

```
docker ps
```

Ahora abre una nueva shell en él:

```
docker exec -it alpine-lab sh
```

Si el contenedor vuelve a detenerse inmediatamente, analiza por qué ocurre según el proceso principal que ejecuta.

Este ejercicio sirve para observar que no todos los contenedores permanecen activos de la misma manera.

Ejercicio 6 — Servidor web Nginx

Objetivo

Crear un servicio accesible desde Windows.

Crea un contenedor llamado:

```
web1
```

Debe:

- ejecutarse en segundo plano;
- utilizar la imagen `nginx`;
- publicar el puerto `8080` de Ubuntu hacia el puerto `80` del contenedor.

Resultado esperado

Desde Windows:

```
http://IP_DE_UBUNTU:8080
```

debe aparecer la página de Nginx.

Pista

Necesitarás:

```
docker run
-d
--name
-p
```

Ejercicio 7 — Gestión del ciclo de vida

Objetivo

Practicar el ciclo de vida de `web1`.

Realiza, en orden:

1. comprobar que está funcionando;
2. detenerlo;
3. comprobar su estado;
4. iniciarlo de nuevo;
5. reiniciarlo;
6. comprobar otra vez su estado.

Comandos disponibles

```
docker ps
```

```
docker ps -a
```

```
docker stop
```

```
docker start
```

```
docker restart
```

Ejercicio 8 — Logs de Nginx

Objetivo

Relacionar una petición web con los logs del contenedor.

1. Ejecuta:

```
docker logs web1
```

1. Abre varias veces desde Windows:

```
http://IP_DE_UBUNTU:8080
```

1. Ejecuta de nuevo:

```
docker logs web1
```

1. Finalmente ejecuta:

```
docker logs -f web1
```

1. Refresca varias veces el navegador.

Pregunta

¿Qué cambia en los logs cada vez que refrescas la página?

Ejercicio 9 — Ejecutar comandos dentro de Nginx

Objetivo

Practicar `docker exec` .

Ejecuta sin entrar en una shell:

```
docker exec web1 hostname
```

```
docker exec web1 pwd
```

Después entra:

```
docker exec -it web1 /bin/bash
```

Dentro ejecuta:

```
ls
```

```
cat /etc/os-release
```

Sal con:

```
exit
```

Pregunta

¿Al ejecutar `exit` se detuvo el contenedor `web1` ?

Compruébalo con:

```
docker ps
```

Ejercicio 10 — Dos servidores Nginx

Objetivo

Comprender que una imagen puede crear varios contenedores.

Mantén:

```
web1
```

en:

```
8080:80
```

Crea además:

```
web2
```

en:

```
8081:80
```

Comprueba ambos desde Windows:

```
http://IP_DE_UBUNTU:8080
```

```
http://IP_DE_UBUNTU:8081
```

Después ejecuta:

```
docker ps
```

Preguntas

- 1. ¿Qué imagen usan `web1` y `web2` ?
- 2. ¿Tienen el mismo `CONTAINER ID` ?
- 3. ¿Utilizan el mismo puerto de Ubuntu?
- 4. ¿Utilizan el mismo puerto interno?

Ejercicio 11 — Investigar un error de puertos

Objetivo

Interpretar un error habitual.

Intenta crear:

```
web3
```

utilizando también:

```
8080:80
```

Preguntas

- 1. ¿Qué error muestra Docker?
- 2. ¿Por qué ocurre?
- 3. ¿Qué cambiarías para solucionar el problema?

Después crea correctamente `web3` utilizando el puerto `8082` .

Ejercicio 12 — Inspección

Objetivo

Explorar información interna de un contenedor.

Ejecuta:

```
docker inspect web1
```

Busca visualmente información relacionada con:

```
IPAddress
Ports
Image
Name
State
```

No es necesario dominar todavía el JSON.

Ejercicio 13 — Estadísticas

Objetivo

Observar recursos consumidos.

Ejecuta:

```
docker stats
```

Observa:

- `web1` ;
- `web2` ;
- `web3` .

Después sal con:

```
Ctrl + C
```

Preguntas

1. ¿Qué columna muestra memoria?
2. ¿Qué columna muestra CPU?
3. ¿Los tres contenedores consumen exactamente lo mismo?

Ejercicio 14 — Copiar un archivo a un contenedor

Objetivo

Practicar `docker cp` .

En Ubuntu Server:

```
echo "Clase de Docker" > mensaje.txt
```

Copia el archivo a:

```
/tmp/mensaje.txt
```

dentro de `web1` .

Después comprueba su contenido utilizando:

```
docker exec
```

y:

```
cat
```

Ejercicio 15 — Eliminar contenedores

Objetivo

Limpiar el entorno.

Detén:

```
web2
web3
```

Elimínalos.

Comprueba después:

```
docker ps -a
```

Mantén `web1` para el ejercicio siguiente.

Ejercicio 16 — Eliminar una imagen

Objetivo

Comprender la dependencia entre imagen y contenedor.

Intenta eliminar:

```
nginx
```

mientras `web1` todavía existe.

Pregunta

¿Por qué Docker puede impedir la eliminación?

Después:

1. detén `web1` ;
2. elimina `web1` ;
3. vuelve a intentar eliminar la imagen.

Ejercicio 17 — Contenedor temporal

Objetivo

Practicar `--rm`.

Ejecuta:

```
docker run --rm hello-world
```

Después:

```
docker ps -a
```

Pregunta

¿Existe ahora un nuevo contenedor `hello-world` detenido?

Explica por qué.

33. Mini laboratorio

Escenario

Eres un técnico junior de una empresa de ingeniería de datos. El equipo necesita levantar rápidamente tres servidores web de prueba en un servidor Ubuntu para realizar pruebas internas.

No se quiere instalar Nginx directamente en Ubuntu. Debe ejecutarse mediante contenedores Docker.

Requisitos

Debes crear:

Contenedor	Puerto Ubuntu	Puerto Nginx
frontend-a	8101	80
frontend-b	8102	80
frontend-c	8103	80

Todos deben utilizar la imagen:

```
nginx
```

y ejecutarse en segundo plano.

Parte A — Despliegue

1. Comprueba si la imagen `nginx` existe localmente.
2. Si no existe, descárgala.
3. Crea los tres contenedores.
4. Comprueba que los tres están activos.

Parte B — Prueba desde Windows

Obtén la IP de Ubuntu:

```
hostname -I
```

Accede desde Windows a:

```
http://IP:8101
```

```
http://IP:8102
```

```
http://IP:8103
```

Los tres deben mostrar Nginx.

Parte C — Administración

Realiza las siguientes acciones:

1. consulta los logs de `frontend-a` ;
2. reinicia `frontend-b` ;
3. detén `frontend-c` ;
4. comprueba el estado de los tres;
5. vuelve a iniciar `frontend-c` ;
6. consulta las estadísticas de los tres.

Parte D — Investigación

Utilizando únicamente comandos Docker, responde:

1. ¿qué imagen utiliza `frontend-a` ?
 2. ¿qué puerto del servidor utiliza `frontend-b` ?
 3. ¿qué contenedores están activos?
 4. ¿qué `CONTAINER ID` tiene `frontend-c` ?
 5. ¿cuánta memoria consume aproximadamente cada contenedor?
-

Parte E — Limpieza

Al terminar:

1. detén los tres contenedores;
2. elimínalos;
3. comprueba que ya no aparecen en `docker ps -a` .

No elimines Docker del servidor.
